

学 号： 2022217549

密 级： 公开

合肥工业大学

Hefei University of Technology

本科毕业设计（论文）

UNDERGRADUATE THESIS



类 型： 毕业设计

题 目： 基于 CNN 的图像超分辨率系统

设计与实现

专业名称： 物联网工程

入校年份： 2022 级

学生姓名： 张磊

指导教师： 李建华 副研究员

学院名称： 计算机与信息学院

完成时间： 2026 年 5 月

合 肥 工 业 大 学

本科毕业设计（论文）

基于 CNN 的图像超分辨率系统设计与实现

学生姓名：张磊

学生学号：2022217549

指导教师：李建华 副研究员

专业名称：物联网工程

学院名称：计算机与信息学院

2026 年 5 月

A Dissertation Submitted for the Degree of Bachelor

**Design and Implementation of a CNN-based Image
Super-Resolution System**

By

Zhang Lei

Hefei University of Technology

Hefei, Anhui, P.R.China

May, 2026

毕业设计（论文）独创性声明

本人郑重声明：所呈交的毕业设计（论文）是本人在指导教师指导下进行独立研究工作所取得的成果。据我所知，除了文中特别加以标注和致谢的内容外，设计（论文）中不包含其他人已经发表或撰写过的研究成果，也不包含为获得合肥工业大学或其他教育机构的学位或证书而使用过的材料。对本文成果做出贡献的个人和集体，本人已在设计（论文）中作了明确的说明，并表示谢意。

毕业设计（论文）中表达的观点纯属作者本人观点，与合肥工业大学无关。

毕业设计（论文）作者签名：

签名日期：2026 年 5 月 18 日

毕业设计（论文）版权使用授权书

本学位论文作者完全了解 合肥工业大学 有关保留、使用毕业设计（论文）的规定，即：除保密期内的涉密设计（论文）外，学校有权保存并向国家有关部门或机构送交设计（论文）的复印件和电子光盘，允许设计（论文）被查阅或借阅。本人授权 合肥工业大学 可以将本毕业设计（论文）的全部或部分内容编入有关数据库，允许采用影印、缩印或扫描等复制手段保存、汇编毕业设计（论文）。

（保密的毕业设计（论文）在解密后适用本授权书）

学位论文作者签名：

指导教师签名：

签名日期：2026 年 5 月 18 日

签名日期：2026 年 5 月 18 日

摘要

图像超分辨率旨在由低分辨率图像恢复出更高分辨率且细节更充分的图像，在老照片修复、移动端图片增强、视频帧增强和数字内容生产等场景中具有较强的应用价值。早期基于插值或稀疏表示的方法难以恢复复杂纹理；深度卷积神经网络和生成式模型显著提升了重建质量，但模型选择、推理速度、显存占用、结果管理和普通用户使用门槛仍然影响其落地。本毕业设计围绕“基于 CNN 的图像超分辨率系统设计实现”展开，在保留 Real-ESRGAN 推理能力的基础上，构建了一个可在本地运行的图像超分 Web 系统。

系统采用前后端分离架构：前端使用 Vue 与 Vite 构建图像处理工作台，提供图片上传、参数配置、原图与结果对比、滑块查看、模型状态展示和任务记录等功能；后端使用 FastAPI 封装统一推理接口，复用 PyTorch、Real-ESRGAN、SwinIR 与 GFPGAN 等模型组件，实现模型权重按需下载、推理器缓存、设备自动选择、分块推理、人脸增强、透明通道处理和结果静态访问。针对端口占用、前端构建、虚拟环境切换等常见部署问题，系统还实现了一键启动脚本，提高了在普通个人计算机上的可用性。

论文首先梳理单图像超分辨率的研究背景与关键技术，分析系统需求和设计目标；随后给出总体架构、功能模块、接口设计和推理流程；最后从功能完整性、异常处理、量化指标、资源占用控制和可维护性等方面进行测试与分析。测试结果表明，系统能够完成 PNG/JPG 图像增强、透明通道保持、模型状态展示、历史任务回看和端口顺延启动等核心流程；在 Set5 与 Set14 数据集的 $4\times$ 测试中，轻量通用模型平均耗时约 0.027 s 和 0.033 s，RealESRGAN_x4plus 平均耗时约 0.185 s 和 0.334 s，输出结果具备可量化评价和可视化对比基础。实现结果表明，该系统能够在本地环境中完成多模型图像超分处理，并在交互体验、模型管理和工程封装方面形成了较完整的毕业设计成果。

关键词：图像超分辨率；Real-ESRGAN；SwinIR；FastAPI；Vue；本地 Web 系统

ABSTRACT

Image super-resolution aims to recover high-resolution images with richer details from low-resolution inputs. It is useful in old photo restoration, mobile image enhancement, video frame enhancement, and digital content production. Traditional interpolation and sparse-representation methods have limited ability to reconstruct complex textures. Deep convolutional networks and generative models have substantially improved restoration quality, but model selection, inference speed, GPU memory usage, result management, and usability remain practical challenges.

This graduation design develops a local Web-based image super-resolution system around the topic of a CNN-based super-resolution application. The system preserves the inference capability of Real-ESRGAN and extends it with an engineering-oriented front-end and back-end architecture. The Vue and Vite front end provides image upload, parameter configuration, original/result comparison, slider-based inspection, model status display, and local task history. The FastAPI back end exposes unified inference APIs and integrates PyTorch, Real-ESRGAN, SwinIR, and GFPGAN components. It supports on-demand model weight downloading, upsampler caching, automatic device selection, tiled inference, face enhancement, alpha-channel handling, and static result access. A one-click launcher is also implemented to handle virtual environment switching, front-end building, browser opening, and port fallback.

This thesis reviews the background and technical foundations of single-image super-resolution, analyzes system requirements, presents the architecture and implementation details, and evaluates the system from the perspectives of functional completeness, exception handling, quantitative metrics, resource control, and maintainability. The tests show that the system supports PNG/JPG enhancement, alpha-channel preservation, model status display, local task history, and automatic port fallback. On the Set5 and Set14 $4\times$ test pairs, the lightweight general model takes about 0.027 s and 0.033 s per image on average, while RealESRGAN_x4plus takes about 0.185 s and 0.334 s, providing both quantitative evaluation and visual comparison evidence. The implementation shows that the system can provide

a usable local workflow for multi-model image super-resolution while maintaining a clear software structure and practical deployment process.

KEYWORDS: image super-resolution, Real-ESRGAN, SwinIR, FastAPI, Vue, local Web system

目 录

1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.3 课题目标与主要工作	3
1.4 论文组织结构	3
2 相关技术基础	5
2.1 单图像超分辨率原理	5
2.2 真实世界退化与模型选择	5
2.3 Real-ESRGAN 与 GFPGAN	6
2.4 SwinIR 与 Transformer 图像复原	7
2.5 推理框架与设备选择	7
2.6 前后端工程框架	7
2.7 本地部署与资源控制技术	8
2.8 本章小结	9
3 需求分析	10
3.1 用户需求	10
3.2 功能需求	11
3.3 用例分析	11
3.4 非功能需求	12
3.5 可行性分析	12
3.6 本章小结	13
4 系统总体设计	14
4.1 总体架构	14
4.2 分层架构与部署边界	15

4.3	数据流与文件组织设计	15
4.4	模块划分	16
4.5	推理流程设计	17
4.6	模型配置设计	18
4.7	接口返回与状态设计	18
4.8	安全与隐私边界设计	19
4.9	本章小结	20
5	系统实现	21
5.1	核心功能闭环实现	21
5.2	后端接口与错误处理实现	21
5.3	推理核心与缓存实现	24
5.4	前端界面与运行效果实现	27
5.5	启动支撑与备用入口实现	30
5.6	本章小结	31
6	系统测试	32
6.1	测试环境与数据	32
6.2	功能测试	32
6.3	边界条件与异常测试	33
6.4	数据集量化测试	34
6.5	性能测试与结果分析	35
6.6	测试结论	36
6.7	工程特点分析	37
6.8	风险分析与改进方案	37
6.9	本章小结	38
7	总结与展望	39
7.1	工作总结	39
7.2	后续展望	39

参考文献	40
致谢	42
附录	43

插图清单

图 2.1	相关技术在系统中的作用关系	8
图 4.1	系统总体架构	14
图 4.2	系统分层架构与部署边界	15
图 4.3	系统数据流设计	16
图 4.4	图像超分推理流程	17
图 4.5	本地运行场景下的安全与隐私边界	20
图 5.1	系统实现中的模块协作关系	21
图 5.2	增强接口核心处理流程	23
图 5.3	系统支持处理的典型输入图像类型	27
图 5.4	增强工作台透明通道图像运行效果	28
图 5.5	模型管理与任务记录运行效果	29

表格清单

表 3.1	系统用户需求分析	10
表 3.2	系统核心用例	11
表 3.3	系统可行性分析	13
表 4.1	系统主要模块及职责	16
表 4.2	系统内置模型配置	18
表 4.3	图像增强接口返回字段设计	19
表 5.1	FastAPI 接口设计	22
表 5.2	后端参数校验策略	24
表 5.3	推理器缓存键设计	26
表 6.1	项目测试资源概况	32
表 6.2	系统功能测试项	33
表 6.3	边界条件与异常测试设计	34
表 6.4	标准数据集量化测试结果	35
表 6.5	标准数据集性能测试结果	35
表 6.6	影响系统性能的主要因素	36
表 6.7	系统风险与改进方案	37
表 7.1	图像增强接口主要参数	43
表 7.2	项目主要目录和文件	44

1 绪论

1.1 研究背景与意义

图像超分辨率（super-resolution, SR）是低层视觉领域中的经典问题，其目标是在给定低分辨率图像的条件下恢复高分辨率图像。若记低分辨率图像为 I_{LR} ，高分辨率图像为 I_{HR} ，则单图像超分辨率可以被描述为学习一个非线性映射 $F(\cdot)$ ，使得

$$\hat{I}_{HR} = F(I_{LR}; \theta), \quad (1.1)$$

其中 θ 表示模型参数， $\hat{I}_{HR}$ 表示系统输出的超分结果。由于从 I_{LR} 到 I_{HR} 的映射存在一对多关系，超分辨率本质上是一个病态逆问题：同一个低分辨率像素块可能对应多种高频纹理，算法不仅要提高分辨率，还要在真实感、结构保持、噪声抑制和推理速度之间取得平衡。

在深度学习之前，常见方法包括双三次插值、邻域回归、稀疏表示和自相似块匹配等。Yu 等^[1] 提出的统一学习框架尝试结合学习型方法与重建型方法，以缓解学习型方法可能引入伪纹理、重建型方法容易平滑细节的问题。随着卷积神经网络的发展，Dong 等^[2] 提出的 SRCNN 将低分辨率到高分辨率的映射统一到端到端 CNN 中，标志着深度学习方法在单图像超分辨率任务中的系统性应用。随后，Kim 等^[3] 提出的 VDSR 通过更深的网络和残差学习提高了重建精度，Zhang 等^[4] 提出的 RCAN 利用通道注意力机制增强高频信息建模能力。这些工作说明，网络结构深度、残差连接和注意力机制对超分质量具有直接影响。

在真实应用中，仅追求峰值信噪比（PSNR）并不能保证视觉观感。SRGAN^[5] 引入对抗学习和感知损失，在四倍放大任务中显著提升了纹理真实感。ESRGAN^[6] 进一步改进生成器结构、对抗损失和感知损失，使生成结果在视觉质量上更稳定。Real-ESRGAN^[7] 则针对真实世界退化复杂、噪声和压缩伪影难以建模的问题，引入高阶退化建模和 U-Net 判别器，使模型更适合普通用户提交的真实低质量图片。SwinIR^[8] 将 Swin Transformer 引入图像复原，在超分辨率、降噪和 JPEG 去伪影等任务中表现出较强的通用性。这些研究共同推动了超分辨率从实验室指标优化走向实际图像增强。

本毕业设计的意义在于将上述研究成果转化为可使用的本地系统。相较于只提供命令行脚本的推理工程，一个面向用户的超分系统需要处理文件上传、模型选择、

参数说明、结果预览、历史记录、错误提示、端口冲突和部署环境等工程问题。对于毕业设计而言，系统不仅要能调用模型，还应体现软件工程中的需求分析、架构设计、模块划分、异常处理和测试验证。因此，本课题围绕现有 Real-ESRGAN 推理能力，设计并实现一个基于 Vue 与 FastAPI 的本地图像超分 Web 系统，以降低普通用户使用深度超分模型的门槛。

1.2 国内外研究现状

从技术路线看，单图像超分辨率大致经历了插值与先验驱动、卷积神经网络驱动、生成式模型驱动和 Transformer/扩散模型驱动几个阶段。传统插值方法计算简单，但无法产生新的高频结构；基于学习的方法通过外部样本或输入图像内部自相似性进行纹理补偿，但依赖特征设计与搜索策略。深度卷积网络出现后，SRCNN^[2] 将特征提取、非线性映射和重建统一到一个网络中，VDSR^[3] 通过深层残差学习扩展感受野，RCAN^[4] 通过通道注意力改善深层网络的信息选择能力。在速度和实时性方面，FSRCNN^[9] 将特征提取放在低分辨率空间中计算以降低推理开销，ESPCN^[10] 则通过亚像素卷积把上采样放到网络末端，更适合实时图像和视频场景。EDSR^[11] 通过去除不必要的批归一化层并扩大残差网络容量提升重建质量，DBPN^[12] 使用迭代上下采样误差反馈增强高频恢复能力。

生成式超分方法的重点从像素误差转向感知质量。SRGAN^[5] 指出，均方误差优化往往得到较高 PSNR，但结果可能缺少高频细节。Johnson 等^[13] 提出的感知损失将特征空间差异引入图像生成与超分任务，为后续基于感知质量的超分方法提供了重要思路。ESRGAN^[6] 在 SRGAN 的基础上采用残差密集块和相对平均判别器，以减少伪影并提升自然纹理表现。Real-ESRGAN^[7] 进一步面向真实世界盲超分任务，其核心思想是通过更复杂的合成退化过程模拟真实图像中的模糊、噪声、压缩和振铃等现象。本系统选择 Real-ESRGAN 作为主要模型来源，正是因为毕业设计面对的是本地用户上传的普通图片，而非固定退化方式的标准数据集。

近几年，Transformer 和扩散模型也被引入图像复原领域。SwinIR^[8] 基于 Swin Transformer 构建图像复原网络，在经典超分、轻量超分和真实世界超分等任务上均给出了较强基线。申报表中提到的轻量化方向也在持续发展，例如 Taylor Neural Network^[14] 尝试以泰勒级数思想增强真实世界超分模型的高阶表征能力，LatticeFormer^[15] 通过门控反馈与轻量结构兼顾参数量和重建质量。扩散模型方向则尝试通

过条件生成过程提升真实感，CDPMSR^[16] 是其中面向单图像超分的代表性探索。考虑到毕业设计的工程边界与本地部署成本，本系统没有训练新的轻量模型或扩散模型，而是在已成熟的 Real-ESRGAN、SwinIR 与 GFPGAN 组件之上完成可用系统封装。

1.3 课题目标与主要工作

本课题的目标不是重新训练一个全新的超分模型，而是围绕“可运行、可交互、可维护”的工程目标完成本地图像超分系统。结合现有项目实施，本文将毕业设计的主要工作概括为以下四个方面。

第一，梳理并封装多模型推理能力。系统将通用照片、二倍放大、轻量通用、动漫、RealESRNet 和 SwinIR 等模型统一纳入后端模型配置，使用户可以根据照片、动漫、视频帧、忠实还原或真实世界复原等不同需求选择模型。

第二，实现本地 Web 交互界面。前端提供上传、拖拽、参数设置、原图与结果对比、滑块查看、任务历史、模型状态和结果下载等功能，使用户不需要记忆命令行参数即可完成图像增强。

第三，设计后端推理服务与工程启动流程。后端使用 FastAPI 暴露健康检查、模型列表和图像增强接口，使用 PyTorch 自动选择 CUDA、MPS 或 CPU 设备，支持模型权重按需下载、推理器缓存、分块处理和异常提示。一键启动脚本负责虚拟环境切换、前端构建、端口检测和浏览器打开。

第四，完成系统测试与论文整理。本文从需求覆盖、接口行为、异常处理、资源约束和可维护性等角度分析系统质量，并结合项目目录、代码结构和测试图片集说明系统的实际完成情况。

1.4 论文组织结构

本文围绕图像超分系统的设计与实现展开，共分为七个部分。第一章说明课题背景、研究现状、项目目标和主要工作，明确本文并非重新训练超分模型，而是完成面向用户的本地化系统封装。第二章介绍单图像超分辨率、真实世界退化建模、Real-ESRGAN、GFPGAN、SwinIR、PyTorch 以及前后端部署等相关技术，为后续系统设计提供理论和工程基础。第三章从用户需求、功能需求、非功能需求和可行性四个角度分析系统边界。第四章给出系统总体架构、数据流、模块划分、推理流程、

模型配置、接口状态和安全隐私边界设计。第五章结合项目代码说明核心功能闭环、接口处理、推理核心、前端交互和启动支撑实现。第六章对测试环境、功能测试、边界测试、质量评价、性能影响和工程风险进行总结。第七章总结毕业设计完成情况，并提出后续改进方向。

2 相关技术基础

本章围绕系统实现所依赖的关键技术展开。首先说明单图像超分辨率的基本退化模型与质量评价指标，其次分析真实世界退化给模型选择带来的影响，再分别介绍 Real-ESRGAN、GFPGAN 和 SwinIR 等模型组件，最后说明 PyTorch、FastAPI、Vue 与本地部署资源控制在本课题中的作用。通过这一章可以看出，论文后续的系统设计并不是孤立的页面开发，而是建立在图像复原模型、推理框架和本地 Web 工程共同约束之上的。

2.1 单图像超分辨率原理

单图像超分辨率通常假设低分辨率图像由高分辨率图像经过模糊、下采样、噪声和压缩等退化过程得到。可简化表示为

$$I_{LR} = D(I_{HR}; \delta), \quad (2.1)$$

其中 $D(\cdot)$ 表示退化过程， δ 表示退化参数。传统监督式超分训练通常使用成对的 I_{LR} 与 I_{HR} 样本学习逆映射。然而，真实图像的退化并不一定符合固定的双三次下采样假设，低质量照片还可能混合运动模糊、传感器噪声、社交平台压缩、多次缩放和锐化伪影。Real-ESRGAN^[7] 所强调的高阶退化建模正是为了解决真实世界退化更复杂的问题。

超分系统的评价可分为客观指标和主观观感两类。PSNR 基于均方误差，其定义为

$$\text{PSNR} = 10 \log_{10} \frac{\text{MAX}_I^2}{\text{MSE}}, \quad (2.2)$$

其中 MAX_I 为图像像素最大值。结构相似性指标 SSIM^[17] 从亮度、对比度和结构三个方面衡量图像质量。在真实场景中，PSNR 与 SSIM 可以衡量结构一致性，但不一定能充分反映纹理真实感。因此，本系统在设计上不将某个单一指标作为用户交互结果的唯一判断标准，而是提供多模型选择和可视化对比，使用户根据具体图片内容进行判断。

2.2 真实世界退化与模型选择

毕业设计中的输入图片来源并不固定，可能来自手机拍摄、社交平台下载、网页截图、压缩视频帧、老照片扫描或动漫素材。这些图像往往同时包含多种退化因素，

例如轻微失焦、运动模糊、JPEG 压缩块、二次缩放、噪声和锐化边缘。若只假设低分辨率图像由标准双三次下采样得到，模型在标准测试集上可能获得较高指标，但面对真实图片时容易出现过度平滑、纹理不自然或边缘振铃等问题。Real-ESRGAN^[7] 的真实世界盲超分思想，正是通过更复杂的合成退化来提升模型对未知退化的适应能力。BSRGAN^[18] 从实际退化组合出发设计盲超分退化模型，也说明真实场景超分不能只依赖固定双三次下采样假设。

从系统角度看，真实退化的不确定性要求前端不能只提供一个固定模型。不同图片内容对模型的偏好并不相同：自然照片更重视材质和边缘的自然感；动漫线稿更重视清晰边界和纯色区域；人像图片还需要关注面部结构稳定性；旧照片或低质量压缩图则需要在降噪和细节恢复之间取舍。因此，本项目没有把所有输入统一交给单个模型，而是通过模型配置表暴露多个模型选项，并在界面上给出适用场景说明。这样的设计虽然增加了前端控件和后端配置复杂度，但更符合真实使用场景。

模型选择还影响计算资源。RRDBNet 类模型通常质量较稳定，但参数量和显存占用相对较高；SRVGG 类轻量模型更适合快速预览；SwinIR 在部分真实世界图像上具有较强恢复能力，但窗口补齐和分块推理使其工程适配更复杂；GFPGAN 则只适合人脸区域明显的图片。论文后续的系统设计与实现，围绕这些模型差异建立统一接口，使用户能在相同交互流程下尝试不同模型。

2.3 Real-ESRGAN 与 GFPGAN

Real-ESRGAN^[7] 是本系统最主要的超分模型来源。与经典 ESRGAN 相比，Real-ESRGAN 更强调真实世界盲超分，其训练过程使用纯合成数据，但退化过程覆盖模糊、噪声、压缩、缩放和常见振铃伪影，因而适合处理来源复杂的用户图片。本系统在后端中保留 Real-ESRGAN 的 RRDBNet 与 SRVGGNetCompact 两类结构，分别对应通用高质量模型与轻量通用模型。

对于包含人脸的低质量图像，仅进行整图超分可能导致面部细节不稳定。GFPGAN^[19] 通过生成式人脸先验进行盲人脸复原，在低质量输入中恢复面部结构、颜色和局部细节。本系统将 GFPGAN 作为可选人脸增强功能，而不是默认处理步骤。这样做的原因是人脸增强会增加推理开销，并且在非人像图片中没有必要启用；将其设计为开关可以让用户在效果和速度之间自行取舍。

2.4 SwinIR 与 Transformer 图像复原

SwinIR^[8] 是基于 Swin Transformer 的图像复原模型，结构上包括浅层特征提取、深层特征提取和高质量图像重建三部分，其中深层模块由残差 Swin Transformer 块组成。相较于纯卷积模型，Transformer 更擅长建模较长距离的依赖关系，但计算和显存开销也更敏感。本系统将 SwinIR-M 与 SwinIR-L 作为高级模型选项，适合用户在机器资源允许的情况下尝试更强的真实世界超分效果。

在工程实现上，SwinIR 与 Real-ESRGAN 的推理接口并不完全一致。项目在 Real-ESRGAN/core.py 中实现了 SwinIRUpsampler 包装器，使其对外暴露与 RealESRGANer 接近的 enhance 接口，并加入窗口尺寸补齐、分块推理和输出缩放逻辑。这个适配层使前端和 FastAPI 接口无需关心底层模型差异，从而保持系统模块边界清晰。

2.5 推理框架与设备选择

系统后端基于 PyTorch^[20] 完成模型加载和张量推理。PyTorch 采用动态图和命令式编程风格，在研究和工程调试中具有较好的可读性和灵活性。本项目在设备选择上依次检测 CUDA、Apple Silicon 的 MPS 和 CPU，使系统能够在不同硬件条件下运行。推理框架层主要关注模型权重、网络结构、设备张量、分块参数和输出保存，这些内容与前端页面显示相互独立。

从工程实现看，推理框架需要解决三个问题。第一，不同模型的构造方式不同，RRDBNet、SRVGGNetCompact 和 SwinIR 不能直接共用同一初始化逻辑；第二，权重文件体积较大，需要按需检查和缓存，避免每次请求重复加载；第三，显存或内存受输入尺寸影响明显，需要通过分块推理和精度选项降低失败概率。因此，论文后续实现部分将推理核心作为独立模块说明，而不是把它简单并入页面开发。

2.6 前后端工程框架

前端采用 Vue 组织交互状态和组件视图，使用 Vite 完成开发构建。Vue 适合描述上传区、参数面板、模型管理、任务记录和结果预览等状态驱动界面；Vite 则用于本地开发和静态资源构建。后端采用 FastAPI 提供接口，统一处理图片上传、模型列表、健康检查和结果访问。FastAPI 的异步接口形式便于把耗时推理放入线程执行，

避免阻塞请求循环。

前后端工程框架的职责与推理框架不同。前端不直接访问权重文件，也不承担模型推理；后端接口负责把用户输入转换为明确的请求参数，并把推理结果整理为前端可显示的 JSON 字段。前后端分离使界面迭代与模型推理可以相对独立，也便于后续扩展批处理、任务队列或远程部署能力。

2.7 本地部署与资源控制技术

图像超分辨率系统与普通 Web 表单系统不同，主要计算发生在模型推理阶段。模型加载需要读取较大的权重文件，推理需要占用显存或内存，输出图像尺寸随放大倍率成平方增长。例如一张宽高均放大 4 倍的图像，像素数量会变为原来的 16 倍，保存和预览都会增加资源消耗。因此，本地部署不能只关注服务是否启动，还需要考虑模型权重、推理设备、分块大小、输出目录和浏览器缓存等工程因素。

本项目采用本地 Web 服务而不是桌面 GUI 的原因有三点。第一，浏览器天然适合展示图片、滑块对比和下载链接，能够减少跨平台界面适配成本。第二，FastAPI 与前端静态资源可以由同一个 Python 进程提供，便于一键启动和演示。第三，本地服务默认绑定 127.0.0.1，上传图片 and 生成结果都保存在本机，避免引入云端存储和账号体系。这样的部署方式牺牲了多人协作能力，但换来了更低的实现复杂度和更明确的数据边界。

资源控制主要通过三类策略实现：设备自动选择、分块推理和模型缓存。设备自动选择使系统能够优先使用 CUDA 或 MPS，在不可用时回退到 CPU；分块推理通过降低单次处理的图像块大小减少峰值显存；模型缓存避免重复加载权重，提升连续处理多张图片时的响应速度。这些策略并不能消除所有性能瓶颈，但能显著提高系统在普通个人电脑上的可运行性。

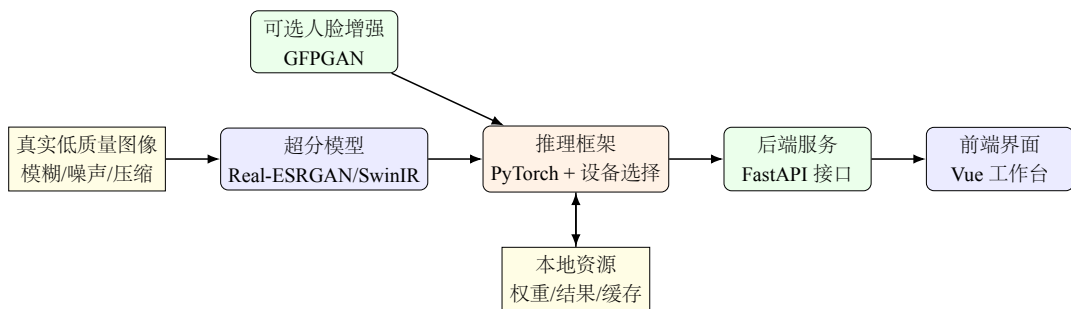


图 2.1 相关技术在系统中的作用关系

图 2.1 概括了本章技术之间的关系：真实低质量图像决定模型需要具备盲超分能力；模型推理依赖 PyTorch 和设备选择；FastAPI 将推理能力封装为接口；Vue 前端再把模型、参数和结果组织成用户可操作的工作流。该关系图也说明，系统设计必须同时考虑算法能力和工程可用性。

2.8 本章小结

本章从算法原理和工程框架两个层面介绍了系统实现所依赖的技术基础。单图像超分辨率提供了问题定义和质量评价依据，真实世界退化解释了选择 Real-ESRGAN、SwinIR 和 GFPGAN 等模型组件的原因，PyTorch、FastAPI、Vue 与本地部署策略则构成系统运行的工程支撑。通过本章分析可以看出，图像超分系统的实现需要同时处理模型效果、推理资源、接口封装和用户交互等多方面约束。

3 需求分析

本章从使用者和系统边界出发分析需求。由于课题成果面向本机运行和毕业设计演示，需求分析不追求复杂业务系统中的多角色权限、订单流程或云端协作，而是关注图像增强这一核心任务能否被普通用户顺畅完成。具体分析包括用户需求、功能需求、用例、非功能需求与可行性五部分。

3.1 用户需求

从使用场景看，本系统的目标用户主要包括需要增强普通照片的个人用户、需要修复插画或动漫素材的内容创作者，以及需要对超分模型进行演示和对比的学生。用户并不一定熟悉命令行、模型权重、显存分块等概念，因此系统需要将复杂参数转化为可理解的界面控件，并在关键位置给出明确反馈。

表 3.1 系统用户需求分析

需求类别	具体需求	设计响应
图像处理	上传单张图片并生成超分结果	支持点击上传和拖拽上传，后端统一执行推理并保存结果
模型选择	针对照片、动漫、轻量推理、人脸场景选择模型	提供模型下拉框和模型管理页，显示模型描述、架构和权重状态
结果查看	对比原图与增强图，查看局部细节	提供对比、滑块、原图、结果四种查看模式，并支持大图弹窗缩放
资源控制	避免显存不足或推理失败	提供分块大小、FP32 精度、人脸增强开关，并对异常给出提示
使用便利	降低环境启动复杂度	一键脚本自动切换虚拟环境、构建前端、选择可用端口并打开浏览器

如表 3.1 所示，本系统的功能设计强调“可解释”和“可恢复”。例如，当端口 7860 被占用时，启动脚本不会直接失败，而是继续尝试后续端口；当前端尚未构建时，后端首页会提示构建命令；当显存不足时，系统提示用户降低分块大小。这些

细节并不改变模型本身，但直接影响毕业设计系统的可用性。

3.2 功能需求

系统功能需求可分为图像增强、模型管理、任务记录和系统运行四部分。图像增强模块负责接收用户图片和参数，向后端发送 `POST /api/enhance` 请求并展示结果。模型管理模块负责读取后端模型列表，显示权重是否已下载、模型架构和默认放大倍率。任务记录模块使用浏览器本地存储保存最近结果，使用户在一次或多次页面刷新后仍能回看历史输出。系统运行模块包括健康检查、设备识别、端口选择、前端构建和旧版 **Gradio** 入口。

本项目没有实现用户登录、云端存储和多人协作，因为这些需求会显著增加系统复杂度，也不是单机图像超分毕业设计的核心目标。系统采用本地运行方式，所有上传图片和生成结果均保存在用户本机，符合毕业设计演示和个人使用的边界。

3.3 用例分析

为了明确系统边界，本文将用户与系统的主要交互归纳为若干用例，如表 3.2 所示。用例分析的重点不是枚举所有按钮，而是说明系统必须支持的完整业务闭环：用户从上传图片开始，经过参数选择和模型推理，最后获得可查看、可下载、可回溯的输出结果。

表 3.2 系统核心用例

用例	触发条件	主要流程	异常或补充处理
图片增强	用户选择或拖拽图片	选择模型和参数，提交到后端，等待结果返回	图片为空、格式异常或推理失败时显示错误信息
模型查看	用户进入模型页	请求模型列表，展示模型架构、倍率和权重状态	权重不存在时提示可在推理时按需下载
环境检查	页面初始化或进入模型页	请求健康检查接口，显示设备和框架版本	GPU 不可用时提示当前使用 CPU 或 MPS
任务回看	用户打开任务记录页	从本地存储读取最近任务，恢复原图和输出图访问链接	结果文件被删除时只保留任务元信息
备用启动	用户指定 Gradio 模式	启动旧版界面，复用共享推理核心	新前端构建失败时可继续演示核心功能

从表 3.2 可以看出，本系统的用例均围绕单机工具展开。系统没有设计管理员、普通用户等多角色权限，也没有引入远程任务队列。这种范围控制使毕业设计能把重点放在模型工程化封装、前后端接口和本地可用性上，而不是扩展到与课题关系较弱的账号系统。

3.4 非功能需求

非功能需求主要体现在可用性、可维护性、兼容性和安全性。可用性要求界面控件清晰、状态反馈及时、结果可下载；可维护性要求模型配置集中、接口边界稳定、前端状态可追踪；兼容性要求系统在 macOS、Windows 和 Linux 上具有基本可运行性；安全性要求默认只监听本机地址 127.0.0.1，避免将本地推理服务暴露给外部网络。

系统还需要考虑资源受限场景。GPU 显存不足是深度超分推理中常见的问题，因此后端支持 tile 分块推理。分块越小，峰值显存越低，但边界处理和总耗时会增加。对于 Apple Silicon 设备，系统优先选择 MPS；若没有可用 GPU，则回退到 CPU。这样的策略使系统在不同硬件上具有可运行性，但不保证所有设备都有相同推理速度。

3.5 可行性分析

从技术可行性看，本项目所需的核心能力均有成熟工具支撑。Real-ESRGAN、SwinIR 和 GFPGAN 已提供可复用的模型结构和权重加载方式；PyTorch 提供跨设备张量推理能力；FastAPI 能够处理文件上传并提供 JSON 接口；Vue 与 Vite 可以快速构建单页交互界面。因此，本系统的主要难点不在于从零实现模型算法，而在于把不同推理组件封装为稳定接口，并使前端能够以一致方式调用。

从经济和环境可行性看，本系统不依赖商业云服务，默认在用户本地运行，开发和演示成本较低。模型权重文件体积较大，但可以按需下载或预先放置到本地权重目录。对于没有高性能 GPU 的设备，系统仍能回退到 CPU，只是推理时间会明显增加。因此，系统可以覆盖毕业设计演示和个人使用需求，但不适合直接承担高并发在线服务。

从进度可行性看，本项目采用“已有模型 + 工程封装”的路线，避免了训练新模型所需的数据准备、长时间训练和指标调参风险。毕业设计周期内，系统实现的

重点是功能闭环和论文论证，包括模型配置、统一接口、前端交互、异常处理、测试图片组织和可运行性验证。该路线与软件开发类毕业设计的要求相匹配。

表 3.3 系统可行性分析

分析维度	有利条件	约束与应对措施
技术可行性	模型、推理框架和 Web 技术均较成熟	通过共享推理核心统一模型差异，降低接口复杂度
环境可行性	本地运行，无需服务器和数据库	设备性能差异较大，通过设备选择和分块推理降低门槛
数据可行性	项目包含多类测试图片，可支持功能验证	不重新训练模型，避免对大规模高低分辨率成对数据的依赖
进度可行性	以工程集成为主，开发任务边界清晰	将用户系统、云存储和批处理作为后续扩展，不纳入核心目标

3.6 本章小结

本章从用户、功能、用例、非功能需求和可行性等方面明确了系统边界。需求分析表明，本系统面向本地单机使用和毕业设计演示，核心任务是让用户能够完成图片上传、模型选择、结果预览、任务回看和异常处理，而不是扩展为复杂的云端多用户平台。通过可行性分析可以看出，已有模型组件和 Web 技术能够支撑系统实现，项目风险主要集中在资源占用、模型权重和环境部署等工程细节上。

4 系统总体设计

本章在需求分析的基础上给出系统总体设计。设计内容按照“架构层次、数据流、模块职责、推理流程、模型配置、接口状态、安全边界”的顺序展开，重点说明系统如何把深度超分模型封装成可交互、可运行、可维护的本地 Web 应用。

4.1 总体架构

系统采用本地 Web 架构，由启动脚本、Vue 前端、FastAPI 后端、模型推理核心、权重目录和结果目录组成。总体结构如图 4.1 所示。

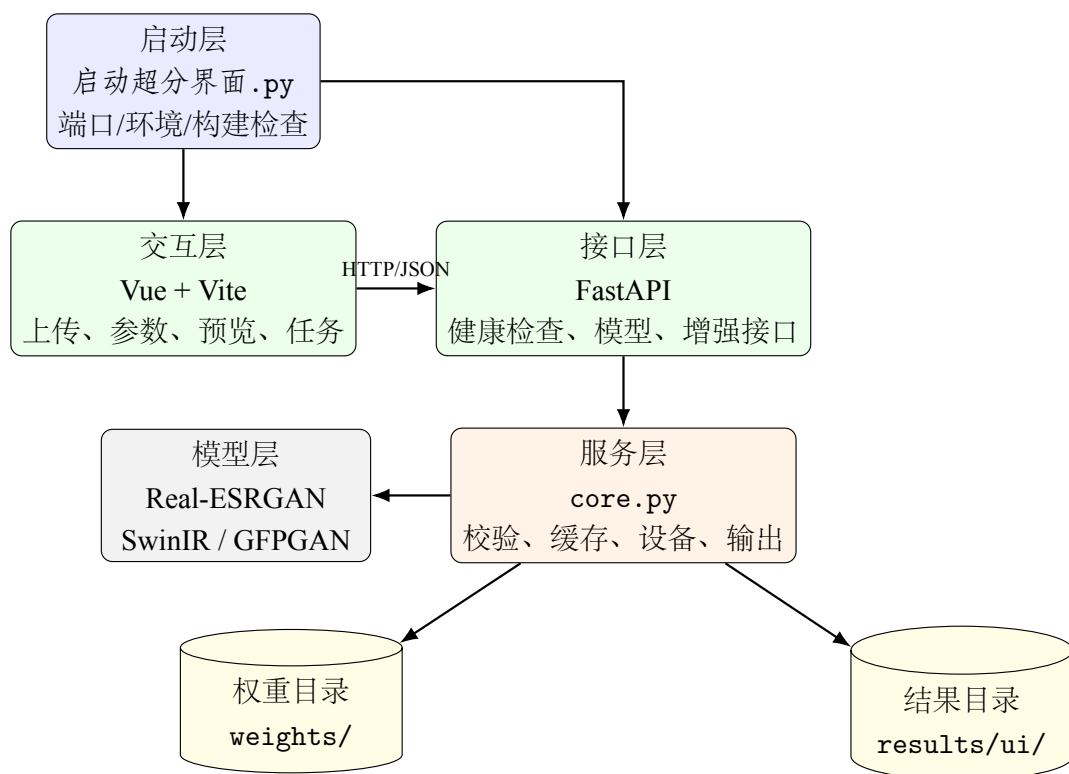


图 4.1 系统总体架构

该架构的核心思想是将“用户交互”和“模型推理”分离。前端只负责收集参数和展示结果，不直接接触模型文件；后端只通过稳定的 API 与前端通信，内部再调用共享推理核心。这样一来，当后续新增模型或修改推理策略时，只需扩展 core.py 的模型配置和适配逻辑，不必大幅改动前端页面。

4.2 分层架构与部署边界

为了进一步说明设计部分中各层之间的依赖关系，本文将系统划分为展示层、接口层、业务服务层、模型推理层和本地资源层，如图 4.2 所示。该划分不是简单的目录罗列，而是对应系统运行时的真实职责：展示层面向用户操作，接口层负责请求入口，业务服务层承接参数校验和任务组织，模型推理层完成图像增强，本地资源层保存权重和输出结果。

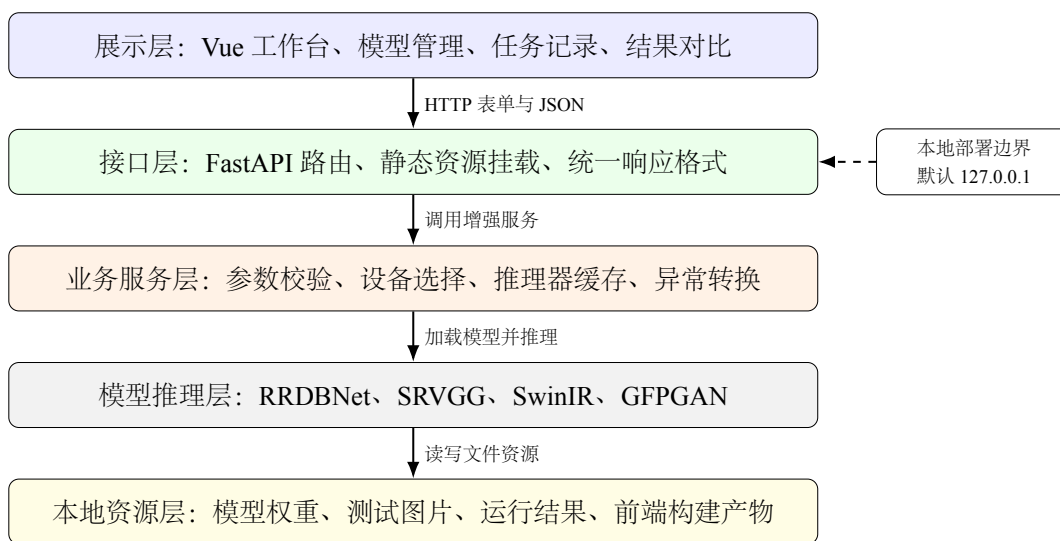


图 4.2 系统分层架构与部署边界

分层设计能够降低系统修改成本。前端页面只依赖接口返回字段，不关心底层模型如何构造；接口层只暴露有限路由，不把模型对象直接传给页面；业务服务层集中处理模型键、分块大小、精度模式和异常信息；模型推理层则复用 Real-ESRGAN、SwinIR 和 GFPGAN 的已有能力。通过这种结构，系统既能体现软件工程设计，也能解释为什么该毕业设计不是简单运行单个推理脚本。

4.3 数据流与文件组织设计

系统的数据流从用户浏览器开始，经过前端表单、后端接口、推理核心和输出目录，最后再回到浏览器预览。其关键不是保存复杂业务数据，而是保证图片文件、模型权重和生成结果之间的路径关系清晰。系统数据流如图 4.3 所示。

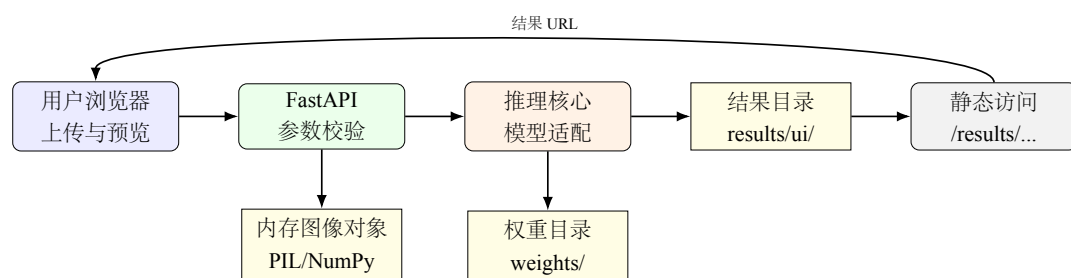


图 4.3 系统数据流设计

输入图片在后端解码后主要以内存对象参与推理，系统不额外设计上传文件长期存储目录。这样做可以减少临时文件清理问题，也能避免用户误以为原图被系统长期保存。输出结果则必须落盘，因为前端需要通过静态 URL 访问增强后的图片，并支持用户下载。输出文件名带有时间戳和模型键，有利于区分不同模型与不同处理时间产生的结果。

文件组织设计遵循“源码、权重、结果、前端构建物分离”的原则。源码目录用于维护系统逻辑；权重目录用于存放可复用模型文件；结果目录用于保存运行时产物；前端构建目录用于提供浏览器页面。这样的组织方式能够避免模型文件和生成结果混入源码，也便于后续清理结果、迁移权重或重新构建前端。

4.4 模块划分

表 4.1 系统主要模块及职责

模块	主要文件	职责说明
启动模块	启动超分界面.py 启动超分界面.bat	检测虚拟环境、检查前端构建、选择端口、启动 Vue 或 Gradio 服务
前端模块	frontend/src/App.vue frontend/src/styles.css	实现页面导航、上传、参数控件、结果预览、滑块对比和任务记录
API 模块	Real-ESRGAN/ vue_server.py	提供健康检查、模型列表和图像增强接口，挂载结果目录和前端静态资源
推理核心	Real-ESRGAN/ core.py	管理模型配置、权重下载、推理器缓存、设备选择、SwinIR 适配和输出保存
备用界面	Real-ESRGAN/ app.py	保留旧版 Gradio UI，复用共享推理核心，作为备用入口

表 4.1 体现了项目改造后的职责边界。原始 Real-ESRGAN 代码主要面向命令行和 Gradio 演示，本项目将模型推理逻辑抽取到 `core.py`，使 Gradio UI 和 Vue/FastAPI UI 都能复用同一套模型配置和推理函数。这种拆分减少了重复实现，也避免了两个界面之间的行为不一致。

4.5 推理流程设计

系统推理流程如图 4.4 所示。用户上传图片后，前端将图片和参数以表单方式提交给后端；后端解码图像并校验模型键；推理核心检查对应权重是否存在，如不存在则按 URL 下载；随后根据模型架构创建或复用推理器，并按用户设置执行放大、降噪、人脸增强和透明通道处理；最后保存结果并返回可访问 URL。

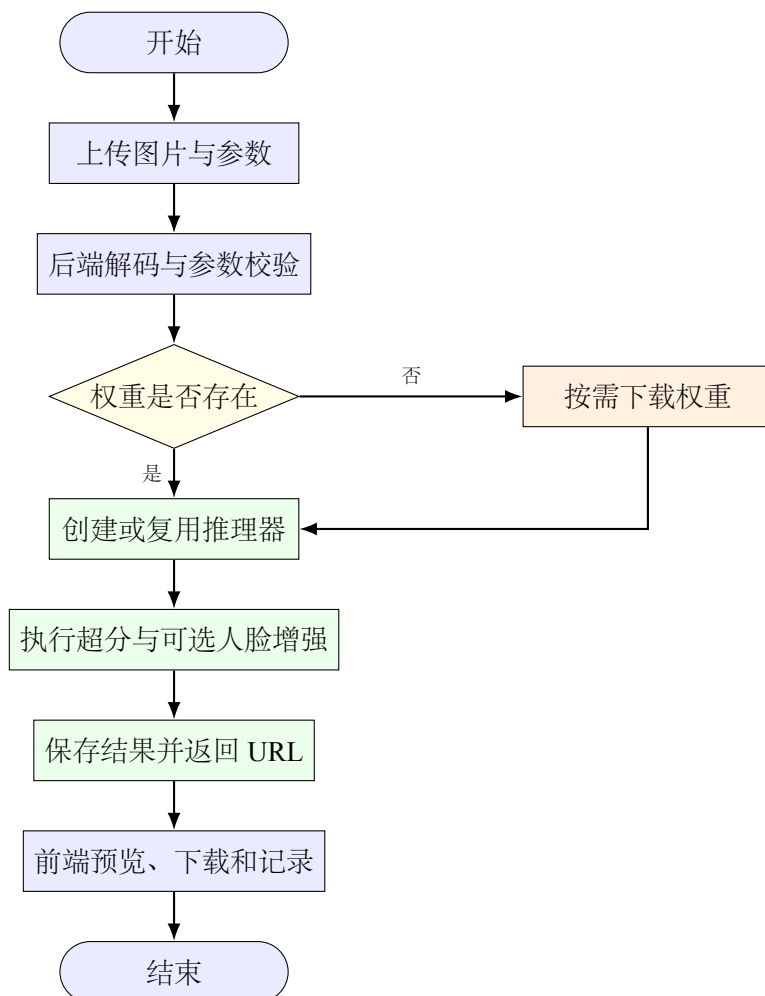


图 4.4 图像超分推理流程

推理器缓存是系统性能优化的关键。深度模型权重加载通常比单次前向推理有

更高的固定开销，若每张图片都重新构造模型，会显著降低交互体验。项目以模型键、降噪强度、分块大小和精度模式等参数构造缓存键，保证同一配置可以复用推理器，而不同配置之间不会相互污染。

4.6 模型配置设计

系统模型列表由 `MODEL_CONFIGS` 统一定义，前端通过 `/api/models` 接口读取。当前模型配置如表 4.2 所示。表中第一列同时给出界面名称和内部键，界面名称面向用户理解，内部键用于后端定位模型权重与网络结构。

表 4.2 系统内置模型配置

界面名称与内部键	架构	放大倍率	适用场景
通用照片 4x <code>RealESRGAN_x4plus</code>	RRDBNet	4×	通用照片、风景、建筑等真实场景
通用照片 2x <code>RealESRGAN_x2plus</code>	RRDBNet	2×	小幅放大，兼顾速度与质量
轻量通用 4x <code>realesr-general-x4v3</code>	SRVGG	4×	轻量通用模型，支持降噪强度调节
动漫插画 4x <code>RealESRGAN_x4plus_anime_6B</code>	RRDBNet	4×	动漫、插画、线稿
动漫视频 4x <code>realesr-animevideov3</code>	SRVGG	4×	动画视频帧与轻量动漫场景
忠实还原 4x <code>RealESRNet_x4plus</code>	RRDBNet	4×	忠实还原，减少生成式纹理
SwinIR 4x <code>SwinIR-M/SwinIR-L</code>	SwinIR	4×	真实世界超分，适合资源较充足环境

这种配置化设计比在接口中写死模型逻辑更容易维护。新增模型时，只要补充标题、描述、权重文件名、下载地址、放大倍率和网络结构参数，前端模型页就能自动显示该模型。对于支持降噪的 `realesr-general-x4v3`，系统会额外显示降噪强度滑块；对于不支持降噪的模型，前端禁用对应控件，避免用户传入无效参数。

4.7 接口返回与状态设计

前后端分离系统需要稳定的数据契约。若后端只返回图片路径，前端虽然可以显示结果，但无法向用户解释模型、耗时和输出尺寸；若后端返回过多内部对象，则

会增加前端解析复杂度，并暴露不必要的实现细节。因此，本项目将接口返回数据控制在“展示结果所需”和“排查问题所需”的范围内。

表 4.3 图像增强接口返回字段设计

字段	类型	设计目的
result_url	字符串	指向生成结果的静态访问地址，用于前端预览和下载
elapsed	数值	记录处理耗时，帮助用户理解不同模型和参数的速度差异
input_size	数组	返回输入图像宽高，用于任务记录和结果说明
output_size	数组	返回输出图像宽高，用于验证倍率是否符合预期
model	字符串	记录实际使用的模型键，便于历史任务回溯
message	字符串	返回成功或失败说明，前端用于状态提示

状态设计上，前端至少需要区分空闲、已选择图片、处理中、成功和失败五种状态。空闲状态展示上传区域；已选择图片后显示原图预览和参数面板；处理中状态禁用重复提交并显示等待反馈；成功状态展示输出图和任务记录；失败状态保留原图和参数，提示用户修改模型、分块大小或输入图片后重试。清晰的状态划分可以避免用户在长时间推理过程中误以为页面无响应。

4.8 安全与隐私边界设计

本系统定位为本地工具，安全边界主要体现在监听地址、文件访问和上传内容处理三个方面。服务默认监听 127.0.0.1，使接口只面向本机浏览器开放，避免未经授权的局域网访问。生成结果仅通过后端挂载的结果目录访问，前端不能任意读取用户文件系统。上传文件在进入推理前由 PIL 解码，系统只处理图片像素数据，不执行上传文件中的任何脚本或外部内容。

隐私方面，系统不需要用户登录，也不会把图片发送到第三方服务器。模型权重下载是唯一需要网络的环节，并且只发生在缺少权重文件时。对于答辩或演示环境，可以提前把常用权重放入 weights/ 目录，以减少现场网络不稳定带来的影响。这样的边界设计与单机毕业设计系统相匹配，也为后续扩展远程部署保留了清晰的改造方向。

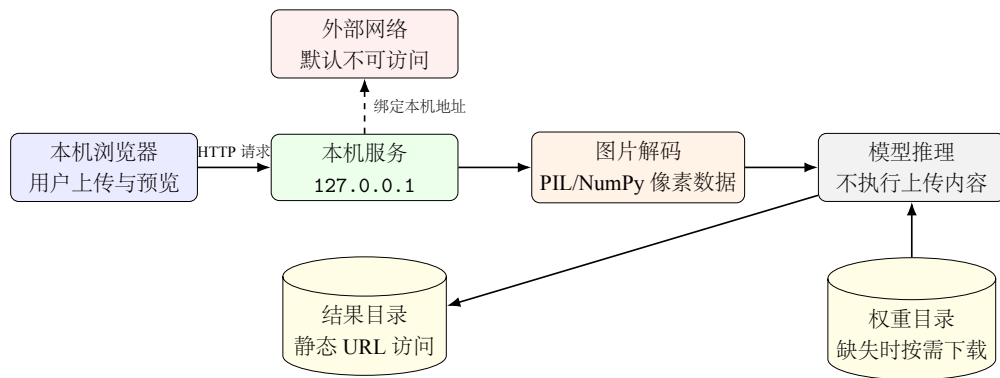


图 4.5 本地运行场景下的安全与隐私边界

4.9 本章小结

本章完成了系统总体设计，给出了本地 Web 架构、数据流、模块划分、推理流程、模型配置、接口状态和安全隐私边界。总体设计的核心是将前端交互、后端接口和模型推理解耦，使用户操作、参数传递、权重管理和结果访问形成清晰链路。该设计既能满足当前毕业设计的运行需求，也为后续增加新模型、批处理任务或远程部署保留了较明确的扩展空间。

5 系统实现

本章围绕图片增强这一核心功能说明系统实现。系统运行时，用户操作由 Vue 前端发起，FastAPI 后端负责接口接入和状态返回，推理核心完成模型加载、权重管理、图像格式处理和结果保存，启动脚本则作为运行支撑模块负责环境、端口和前端构建准备。为了保持实现层次清晰，本章将实现内容归并为接口处理、推理核心、前端运行效果和启动支撑四类，先说明核心功能闭环，再展开各模块细节。整体实现关系如图 5.1 所示。

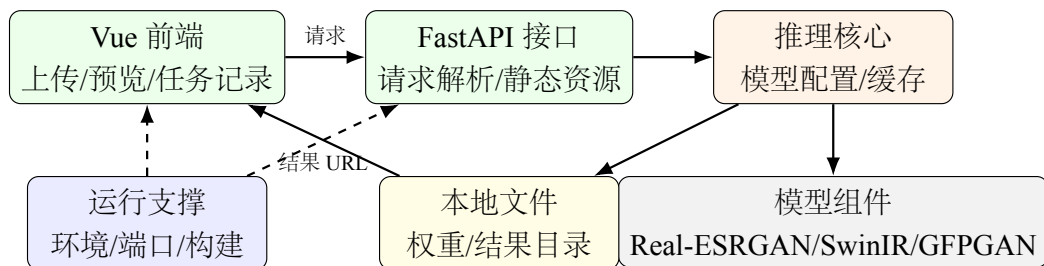


图 5.1 系统实现中的模块协作关系

5.1 核心功能闭环实现

本系统的核心功能是完成一次从图片上传到结果下载的超分任务。该闭环包括四个关键步骤：首先，前端读取用户选择的图片并同步模型、倍率、分块大小、人脸增强和精度模式等参数；其次，后端接收表单数据，完成图片解码、模型键校验和参数整理；再次，推理核心根据模型配置加载或复用推理器，执行超分、透明通道处理和可选人脸增强；最后，系统将输出结果保存到本地结果目录，并把结果 URL、输入输出尺寸、耗时和状态信息返回前端。

围绕这一闭环，前端、后端和推理核心形成了较清晰的模块边界。前端不直接操作模型文件，而是通过接口提交任务；后端不承担具体神经网络结构细节，而是把增强请求转交给 `core.py`；推理核心不关心页面布局，只负责模型、权重、设备、缓存和结果文件。启动脚本虽然是用户运行系统的入口，但在实现说明中属于运行支撑模块，核心业务仍是图像增强请求的处理链路。

5.2 后端接口与错误处理实现

后端位于 `Real-ESRGAN/vue_server.py`，采用 FastAPI 构建。接口设计见表 5.1。

表 5.1 FastAPI 接口设计

方法	路径	功能说明
GET	/api/health	返回当前推理设备、PyTorch 版本和加速后端信息，用于模型页显示运行环境
GET	/api/models	返回模型键、标题、描述、架构、放大倍率、是否支持降噪和权重是否就绪
POST	/api/enhance	接收上传图片 and 推理参数，在线程池中执行超分，返回结果 URL、耗时和图像尺寸
GET	/results/...	作为静态资源访问生成结果，供前端预览与下载

上传图片通过 `UploadFile` 接收，并使用 `PIL` 解码为 `NumPy` 数组。对于 RGB、RGBA 和灰度图，系统保留原有通道语义；对于其他模式，系统转换为 RGB 或 RGBA。推理过程使用 `asyncio.to_thread` 放到线程中执行，避免阻塞 FastAPI 的事件循环。推理完成后，后端将输出路径转换为相对 URL，并附加文件修改时间作为查询参数，避免浏览器缓存旧图。

异常处理遵循“用户可理解”的原则。若上传文件为空，接口返回 400；若模型键无效，核心逻辑抛出 `ValueError` 并转换为 400；若推理过程中发生运行时错误，则返回 500，并在前端显示错误信息。对于显存不足等常见错误，旧版 Gradio 界面也做了专门提示，建议用户降低分块大小。

增强接口是系统最核心的功能入口，其内部处理流程如图 5.2 所示。该流程把一次用户点击“开始超分”后的处理步骤拆为文件接收、参数校验、模型准备、异步推理、结果保存和前端反馈六个环节，能够更清楚地说明模块细节。

图 5.2 中的错误分支对系统可用性很重要。若文件无法解码或模型键不存在，系统不应继续加载权重；若推理阶段发生显存不足或权重损坏，后端需要把底层异常转换为用户能够理解的提示。这样既能减少无效计算，也能让前端在失败后保留原图和参数，方便用户调整分块大小、精度模式或模型后重新提交。

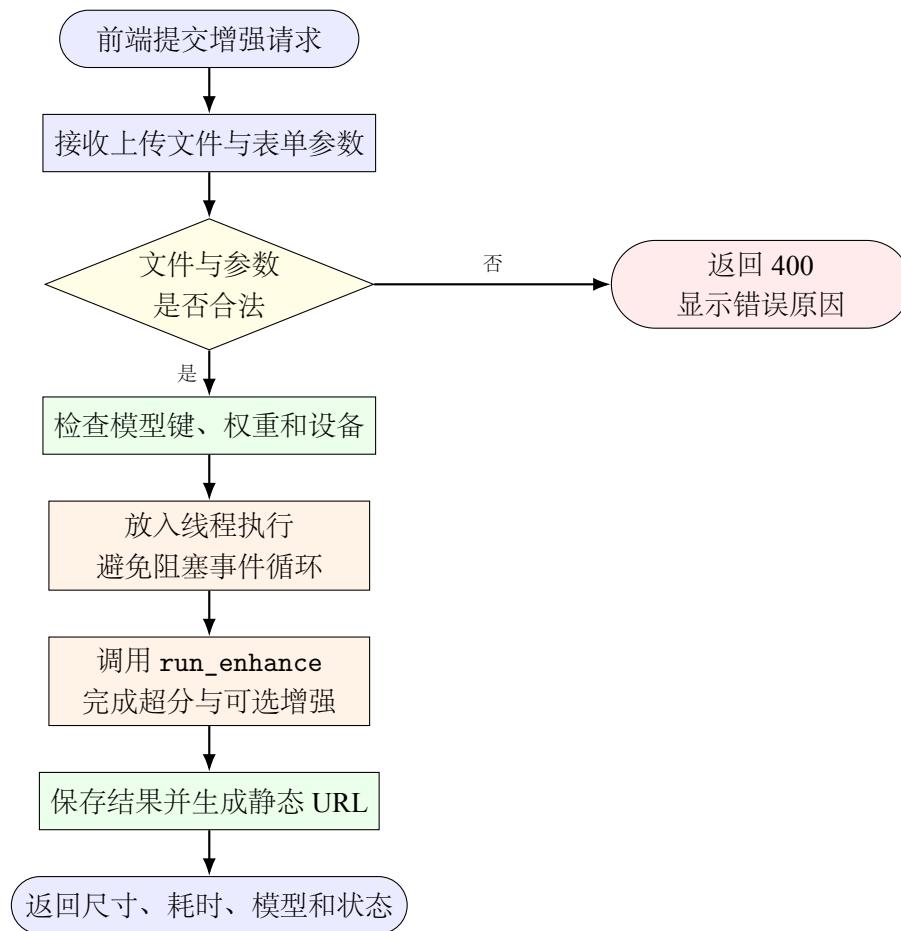


图 5.2 增强接口核心处理流程

接口参数校验与错误处理

图像增强接口的输入既包含文件，也包含模型、倍率、分块大小、降噪强度和人脸增强开关等参数。若前端传入的参数缺少校验，错误会延迟到模型推理阶段才暴露，导致用户等待较长时间后才得到失败信息。因此，后端在进入推理核心前先完成参数解析和基本校验，包括文件是否为空、模型键是否存在、数值参数是否能够转换、图片是否能被 PIL 正常解码等。

表 5.2 后端参数校验策略

校验对象	可能问题	处理方式
上传文件	文件为空、扩展名与内容不一致、无法解码	返回明确错误，不进入模型推理阶段
模型键	前端缓存过期或用户构造非法请求	与模型配置表匹配，不存在则返回参数错误
倍率参数	数值为空、超出合理范围或类型错误	使用默认值或返回错误提示，避免输出尺寸异常
分块大小	分块过小导致速度过慢，过大可能显存不足	接收用户设置，并在失败提示中建议减小分块
人脸增强	非人像图片启用后收益较低且耗时增加	作为可选布尔开关，由用户自行决定是否启用

错误处理遵循“尽早失败、保留上下文”的原则。参数错误应尽早返回 400，避免无意义地加载模型；运行时错误保留前端输入状态，使用户可以调整参数后再次提交；模型下载失败则提示权重文件未就绪。前端不直接显示 Python 堆栈，而是展示简化后的错误说明，这样既降低普通用户理解成本，也避免暴露过多内部路径信息。

5.3 推理核心与缓存实现

Real-ESRGAN/core.py 是项目的核心文件。它首先定义模型配置、权重目录、结果目录、设备选择函数和模型构造函数。模型构造根据架构字段分为三类：RRDBNet、SRVGGNetCompact 和 SwinIR。其中，RRDBNet 用于 RealESRGAN 系列和 RealESRNet 模型；SRVGGNetCompact 用于轻量通用模型与动漫视频模型；SwinIR 用于 SwinIR-M 和 SwinIR-L。

权重下载由 ensure_weight 负责。如果本地 weights/ 目录中不存在对应文件，

则调用 `load_file_from_url` 下载。对于支持降噪的模型，系统会同时管理普通权重和强降噪权重，并根据用户给定的降噪强度进行插值。人脸增强由 `GFPGANer` 封装，只有用户打开人脸增强开关时才会初始化。

推理函数 `run_enhance` 的主要逻辑见算法 1。

算法 1 图像增强核心流程

输入： 输入图像 I ，模型键 m ，输出倍率 s ，分块大小 t ，增强选项 o

输出： 输出图像路径与处理状态

```

1: if  $I$  为空或  $m$  不在模型配置表中 then
2:   return 参数错误信息
3: end if
4:  $cfg \leftarrow$  根据  $m$  读取模型配置
5: if  $cfg$  对应的权重文件不存在 then
6:   调用权重下载函数获取模型权重
7: end if
8:  $key \leftarrow (m, device, denoise, t, fp32)$ 
9: if  $key$  不在推理器缓存中 then
10:  按  $cfg$  创建网络结构并加载权重
11:  将新推理器写入缓存
12: end if
13: 将  $I$  转换为模型输入格式
14: 调用缓存推理器执行  $s$  倍超分
15: if  $I$  包含透明通道 then
16:  对透明通道执行同步放大并合并输出
17: end if
18: if  $o$  启用人脸增强 then
19:  调用 GFPGAN 对输出图像进行局部增强
20: end if
21: 保存输出图像到 results/ui/
22: return 输出路径、输入输出尺寸、耗时和处理状态

```

该流程的工程重点是“先校验、再加载、后推理”。模型加载属于高开销操作，因此缓存策略对交互体验很重要。输出文件名包含时间戳和模型键，可以避免覆盖历史结果，并便于任务记录回溯。

权重管理与推理缓存

模型权重管理是系统稳定性的关键。若所有权重都要求用户手动下载，系统使用门槛会明显提高；若每次启动都检查并下载全部权重，又会增加首次运行时间和网络依赖。因此，本项目采用按需检查策略：只有用户选择某个模型并开始推理时，后端才检查对应权重是否存在；若缺失则调用下载函数获取权重；若已存在则直接复用本地文件。

推理器缓存以模型和关键参数为维度。缓存粒度过粗会导致不同参数互相影响，例如降噪强度或分块大小变化后仍复用旧推理器；缓存粒度过细则会造成重复加载权重，浪费内存。因此，系统将模型键、设备、降噪强度、分块大小、边缘填充和精度模式纳入缓存键。缓存设计见表 5.3。

表 5.3 推理器缓存键设计

缓存维度	示例	作用
模型键	RealESRGAN_x4plus	区分不同网络结构和权重文件
运行设备	CUDA、MPS、CPU	避免跨设备复用已加载模型
降噪强度	0.5、1.0	支持轻量通用模型的不同降噪配置
分块参数	tile、tile pad	保证显存控制参数与推理器状态一致
精度模式	FP16 或 FP32	兼容不同设备的数值稳定性需求

缓存机制提高了连续处理体验。例如用户对多张照片使用同一模型和相同参数时，系统不需要反复构建网络和加载权重，后续请求可以直接进入图像推理阶段。该策略也使前端任务记录更有意义，因为用户可以在同一会话中快速比较多张图片或多个模型效果。

图像格式与透明通道处理

用户上传的图片格式并不统一。常见 JPG 图片只包含 RGB 通道；PNG 图片可能包含透明通道；部分灰度图只有单通道；WebP 图片可能由浏览器或 PIL 解码为不同模式。若系统直接把所有输入强制转换为 RGB，透明图像会丢失 alpha 信息；若完全保留原模式，部分模型又无法处理非三通道输入。因此，后端在进入模型前需要进行统一且可解释的格式转换。

本项目的处理原则是：普通 RGB 图片保持 RGB；带透明通道的图片保留 RGBA

语义，并在推理流程中处理透明区域；灰度图转换为可被模型接受的格式；其他模式则转换为 RGB 或 RGBA。对于图像超分任务而言，透明通道不包含纹理内容，但它决定了输出图像在前端或设计软件中的叠加效果，因此不能简单丢弃。系统在输出保存时尽量保持输入通道语义，使插画、图标和透明素材也能被较处理好。

格式处理还影响输出文件类型。若输入包含透明通道，输出优先保存为支持透明度的格式；若输入为普通照片，则保存为常见图片格式即可。这样的策略兼顾了兼容性和结果可用性，避免用户下载后发现背景变黑或透明区域消失。

5.4 前端界面与运行效果实现

前端主体位于 `frontend/src/App.vue`。界面采用侧边导航与工作区布局，包含增强工作台、模型管理和任务记录三页。增强工作台提供图像上传、参数面板和预览区；模型管理页显示后端模型状态、运行设备、PyTorch 和加速后端信息；任务记录页展示最近处理结果并支持恢复到工作区。

前端状态主要包括当前页面、模型列表、健康信息、选中文件、原图预览、结果 URL、任务记录、查看模式和处理状态。任务记录使用 `localStorage` 保存，键名为 `imageplus_jobs`。这种设计不会把图片上传到远程服务器，符合本地工具的定位。查看模式包括“对比”“滑块”“原图”和“结果”。其中滑块模式通过 CSS `clip-path` 控制原图覆盖区域，并根据鼠标或触摸位置更新滑块比例。



图 5.3 系统支持处理的典型输入图像类型

图 5.3 展示了项目中可用于测试的典型输入。系统并不限制用户只处理某一类图像，而是通过模型选择和参数配置覆盖人物照片、自然图像、标准测试图和城市结构图等不同场景。

系统运行效果展示

为验证前端界面与后端推理接口能够形成完整闭环，本文补充系统在本地环境中运行时的界面截图。运行服务通过浏览器访问本机地址，页面左侧为功能导航，主工作区负责图像预览与结果对比，右侧参数栏负责模型、倍率、分块和增强选项配置。增强工作台的实际运行效果如图 5.4 所示。

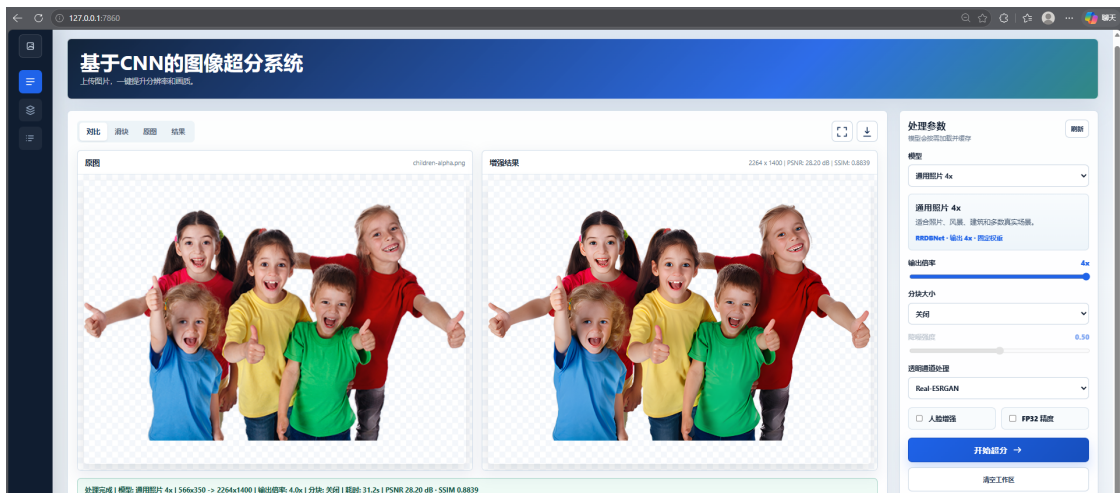
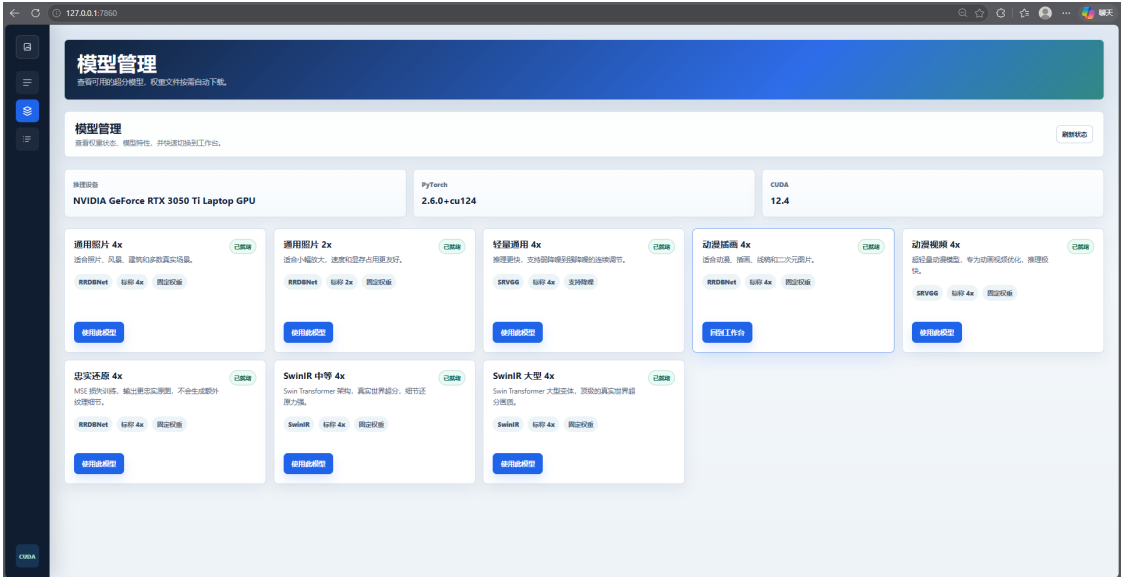


图 5.4 增强工作台透明通道图像运行效果

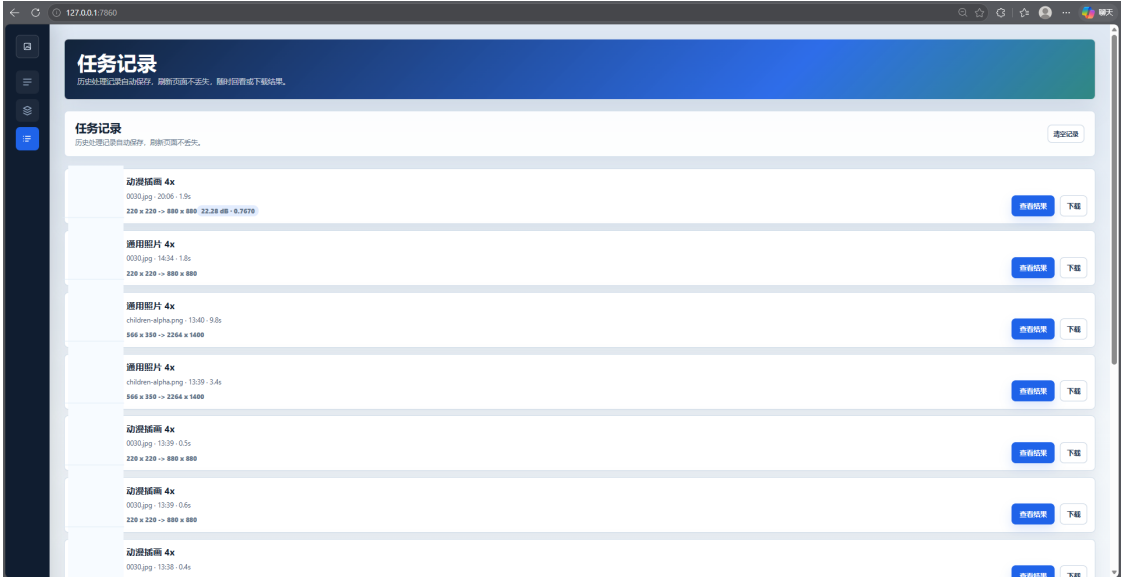
从图 5.4 可以看出，系统能够在同一页面中展示原图、增强结果、输出尺寸、PSNR、SSIM、耗时和最近任务。透明通道示例表明系统在处理 PNG 图像时能够保持透明背景语义，避免输出结果出现异常底色；同时，右侧参数栏完整保留模型选择、倍率、分块、人脸增强和精度模式等关键控制项，能够反映前后端接口的实际联动效果。

除增强工作台外，系统还提供模型管理和任务记录页面。模型管理页用于展示运行设备、PyTorch 与加速后端版本以及各模型权重状态；任务记录页用于保存最近处理结果，便于用户回看、下载和继续对比。相关运行界面如图 5.5 所示。

这些运行截图与前文需求分析中的模型查看、环境检查、图片增强和任务回看用例相对应。系统界面没有把模型权重、设备信息和历史结果隐藏在命令行中，而是以可视化方式暴露给用户，使毕业设计成果更接近可演示、可操作的完整应用。



(a) 模型管理与运行环境状态



(b) 历史任务记录列表

图 5.5 模型管理与任务记录运行效果

前端任务状态与交互细节

前端交互的重点是让用户知道“当前能做什么”和“系统正在做什么”。当用户尚未上传图片时，工作台突出上传区域；用户选择图片后，界面显示原图预览和参数控件；提交任务后，按钮进入处理中状态，避免重复提交；任务成功后，结果图、下载链接和任务记录同步更新；任务失败后，界面保留原图和参数，便于用户降低分块大小或切换模型后重试。

任务记录使用浏览器本地存储实现，而不是写入后端数据库。该方案的优点是实现简单、无需用户登录、符合本地工具定位；不足是记录只在当前浏览器环境中有效，且结果文件被删除后历史记录无法恢复图片内容。对于毕业设计系统，这种取舍是合理的，因为任务记录的主要作用是辅助当前会话中的结果对比，而不是承担长期资产管理。

结果查看提供对比、滑块、原图和结果四种模式。对比模式适合整体观察构图和颜色变化；滑块模式适合检查局部细节差异；原图模式便于确认输入质量；结果模式便于下载前检查最终输出。四种模式共用同一组图片状态，避免重复保存数据，也使界面逻辑保持一致。

5.5 启动支撑与备用入口实现

在核心增强流程之外，项目根目录的 `启动超分界面.py` 负责降低系统运行门槛。脚本首先检查项目根目录下是否存在 `.venv`，若存在且当前解释器不是虚拟环境中的 `Python`，则通过 `os.execv` 重新启动自身。这样用户双击或命令行运行脚本时，系统可以自动使用项目依赖环境，减少“装了依赖但运行时找不到包”的问题。

端口选择采用“首选端口加顺延扫描”的策略。默认端口为 `7860`；若被占用且用户没有指定 `--strict-port`，脚本会继续尝试后续 `50` 个端口，并在终端输出实际访问地址。该策略适合开发和演示环境，因为用户可能已经启动过其他 `Gradio` 或 `Web` 服务。若用户希望固定端口，则可以使用 `--strict-port` 使端口冲突直接报错。

前端构建检查也是启动支撑的重要职责。脚本会比较前端构建产物与源文件的修改时间，判断是否需要执行 `npm run build`。首次构建时，如果 `node_modules` 不存在，脚本会先执行 `npm install`。这种做法使普通用户只需执行一个入口命令即可运行系统，同时仍保留重建前端和跳过构建两类参数，供开发者控制构建行为。

旧版 Gradio 入口保留

虽然新版界面采用 Vue 与 FastAPI，但项目仍保留 `Real-ESRGAN/app.py` 作为旧版 Gradio UI。这样做有两个原因：一是 Gradio 对快速演示和模型调试仍然方便；二是保留备用入口可以降低系统改造风险。当 Vue 前端出现构建问题或浏览器兼容问题时，用户可以通过 `--gradio` 参数启动旧界面继续使用核心推理功能。

保留旧入口并不意味着维护两套推理逻辑。Gradio UI 同样调用共享增强函数，模型配置、权重下载、结果保存和设备选择都与新后端一致。因此，旧界面更像是一个备用视图，而不是独立分支。

5.6 本章小结

本章围绕项目代码说明了系统实现过程。系统首先以图片增强任务为主线，说明前端、后端、推理核心和结果目录之间的协作关系；随后把后端接口与错误处理合并说明，把推理函数、权重管理、缓存键和图像通道处理归入同一实现链路，再结合运行截图说明前端界面、任务状态和结果展示方式。最后，论文将启动脚本与旧版 Gradio 入口作为运行支撑进行说明，避免实现章节只停留在启动脚本层面。

从实现结果看，本章体现了三个工程重点。第一，增强接口不是简单传递文件，而是包含参数校验、异常转换、异步推理和静态结果访问等环节；第二，推理核心不是单模型调用，而是通过配置表、缓存键和设备选择统一管理 Real-ESRGAN、SwinIR 与 GFPGAN；第三，前端界面不是单张图片预览，而是包含对比、滑块、模型管理和任务记录等完整交互。通过这些实现，系统从单一模型推理脚本扩展为具有完整交互流程和工程可维护性的本地应用。

6 系统测试

本章围绕系统是否达到毕业设计要求进行测试说明。测试内容分为测试环境与数据、功能测试、边界条件测试、数据集量化测试、性能测试和风险改进几个部分，既关注正常输入能否得到预期输出，也关注异常输入、资源不足和运行环境变化时系统是否能给出明确反馈。

6.1 测试环境与数据

项目中包含 `test_images/` 测试图片目录，大小约 399 MB，包含 805 个图像文件，覆盖 BSDS100、BSDS200、General100、Set5、Set14、T91、historical、manga109 和 urban100 等常见超分测试图像集合。Arbelaez 等^[21] 发布的 BSDS500 数据集包含 500 张自然图像，并给出训练、验证和测试划分。本章选取 Set5 和 Set14 的 4× 低分辨率图像作为量化测试对象，结合 PSNR、SSIM 和推理耗时评价系统在标准数据集上的基本表现。

表 6.1 项目测试资源概况

资源	当前项目状态	用途
<code>test_images/</code>	约 399 MB，805 个图像文件	功能验证、视觉对比、指标评测
<code>Real-ESRGAN/weights/</code>	已包含部分常用权重，缺失权重可自动下载	模型推理与快速启动
<code>frontend/dist/Real-ESRGAN/results/ui/</code>	已存在构建产物运行时自动生成	支持一键脚本直接启动新版界面保存用户超分结果并供前端访问

6.2 功能测试

系统测试以功能覆盖和异常路径为主。由于深度超分推理依赖模型权重、GPU 环境和输入尺寸，测试重点放在接口闭环、状态反馈和资源控制是否合理。测试方法采用黑盒测试与代码路径核对相结合的方式：黑盒测试关注用户能否通过界面完成任务，代码路径核对关注每个测试项在后端和前端是否有明确实现。每个测试项均明确输入或操作方式，并给出可观察的预期输出，如表 6.2 所示。

表 6.2 系统功能测试项

测试项	输入与操作	预期输出
启动测试	运行 <code>python</code> 启动超分界面 <code>.py</code>	自动选择端口，构建或复用前端，启动 <code>Vue</code> 服务
健康检查	访问 <code>/api/health</code>	返回设备类型、设备名称、 <code>PyTorch</code> 和加速后端信息
模型列表	访问 <code>/api/models</code>	返回模型描述、架构、倍率和权重状态
图片增强	上传 <code>PNG/JPG</code> 图片并选择模型	返回结果 <code>URL</code> 、输入输出尺寸、模型信息和耗时
透明通道	上传 <code>RGBA</code> 图片并选择透明通道处理方式	输出图像保留透明通道语义
异常输入	上传空文件或无效图片	后端返回明确错误，前端显示失败原因
端口占用	默认端口被占用时启动系统	自动尝试后续端口，终端输出真实访问地址

从项目实现看，上述测试项均有对应代码路径。尤其是模型列表与健康检查接口，使前端能够在不执行推理的情况下先判断运行环境；这对大模型系统很重要，因为用户不应等到推理失败后才知道权重未下载或设备不可用。

6.3 边界条件与异常测试

深度图像处理系统的异常往往不是传统表单错误，而是由输入图片、模型权重、设备资源和文件系统共同触发。因此，仅测试正常上传和正常输出是不够的，还需要覆盖边界条件。本文将边界测试分为输入边界、资源边界、模型边界和运行环境边界，具体见表 6.3。

表 6.3 边界条件与异常测试设计

测试类别	测试场景	关注点
输入边界	空文件、损坏图片、灰度图、透明 PNG	后端能否识别错误输入，是否保留透明通道语义
参数边界	分块大小过大或过小，倍率参数异常	是否给出可理解提示，是否避免异常参数进入推理核心
模型边界	权重缺失、模型键不存在、网络下载失败	是否触发按需下载，失败时是否能提示权重问题
资源边界	大尺寸图片、GPU 显存不足、CPU 回退	是否建议降低分块大小，系统是否仍可运行
环境边界	默认端口被占用、前端未构建、虚拟环境未激活	启动脚本是否能顺延端口、构建前端或切换解释器

边界测试的价值在于验证系统能否“失败得足够清楚”。对于普通用户而言，推理失败本身并不可怕，真正影响体验的是失败原因不明确。例如显存不足时，如果系统只显示底层运行时错误，用户很难知道应降低分块大小；如果权重下载失败但界面仍提示模型不可用，用户也难以判断是模型不存在还是网络问题。因此，异常路径的提示质量是本系统测试的重要内容。

6.4 数据集量化测试

图像超分结果的质量评价具有一定复杂性。PSNR 和 SSIM 适合衡量成对测试图像的结构一致性，其中 SSIM^[17] 从结构相似角度补充了单纯像素误差的不足。然而，真实世界盲超分常常没有标准高分辨率真值图，且生成式模型可能以牺牲像素级一致性换取更自然的纹理。Ledig 等^[5] 关于 SRGAN 的研究和 Wang 等^[6] 关于 ESRGAN 的研究均说明，视觉真实感和像素误差指标之间存在张力。

为回应系统测试中的量化要求，本文在 Set5 和 Set14 的 LRbicx4/GTmod12 图像对上进行补充测试。测试时使用系统后端的 `run_enhance` 作为统一入口，输出结果与高分辨率真值图对齐后计算 PSNR 和 SSIM。指标计算采用 RGB 通道，去除 4 像素边界，以减少边缘插值误差对结果的影响。测试环境为当前本地开发环境，推理设备为 Apple Silicon GPU (MPS)，分块大小设为 256，模型预热后记录单张推理耗时。

表 6.4 标准数据集量化测试结果

模型	数据集	数量	平均 PSNR/dB	平均 SSIM
realesr-general-x4v3	Set5	5	24.486	0.7475
realesr-general-x4v3	Set14	14	23.245	0.6562
RealESRGAN_x4plus	Set5	5	24.243	0.7164
RealESRGAN_x4plus	Set14	14	23.414	0.6340

从表 6.4 可以看出，两个模型在 Set5 和 Set14 上都能够完成 $4\times$ 输出，并取得较稳定的结构相似度。其中，轻量模型 realesr-general-x4v3 在 Set5 上的 SSIM 略高，说明其在小规模标准图像上对结构保持更友好；RealESRGAN_x4plus 在 Set14 上的 PSNR 略高，但 SSIM 较低，反映出生成式增强模型可能产生更强纹理，同时未必在像素级指标上占优。因此，论文不能只用单一指标判断模型优劣，还需要结合视觉效果、处理速度和实际场景选择模型。

6.5 性能测试与结果分析

本毕业设计没有把性能指标作为唯一评价目标，但系统必须解释哪些因素会影响运行速度和显存占用。图像超分的计算量与输入尺寸、放大倍率、模型结构和分块大小有关。输入图像越大，输出像素数量越多；放大倍率越高，后处理和保存耗时越明显；模型越复杂，加载和推理时间越长；分块越小，峰值显存越低，但分块调度次数会增加。

表 6.5 标准数据集性能测试结果

模型	数据集	平均耗时/s	最小耗时/s	最大耗时/s
realesr-general-x4v3	Set5	0.027	0.012	0.042
realesr-general-x4v3	Set14	0.033	0.011	0.061
RealESRGAN_x4plus	Set5	0.185	0.109	0.372
RealESRGAN_x4plus	Set14	0.334	0.111	0.592

表 6.5 表明，轻量模型的平均耗时明显低于 RealESRGAN_x4plus。原因在于 realesr-general-x4v3 采用更轻量的 SRVGG 结构，而 RealESRGAN_x4plus 使用更深的 RRDBNet，具有更强的真实图像增强能力，但计算开销也更高。对于毕业设计系统而言，这一结果说明模型管理页中区分“轻量通用”和“通用照片”是有必要

的：前者适合快速预览和普通设备，后者适合用户愿意等待更长时间以换取更强细节增强的场景。

表 6.6 影响系统性能的主要因素

因素	对系统的影响	系统应对方式
输入尺寸	大图会显著增加推理时间和输出文件大小	提供分块推理参数，必要时提示用户降低分块大小
模型结构	SwinIR 和 RRDBNet 等模型计算开销不同	在模型页显示模型说明，引导用户按场景选择
设备类型	CUDA、MPS 和 CPU 的推理速度差异明显	后端自动选择可用设备，并在健康检查中展示
权重加载	首次使用模型需要加载或下载权重	使用按需下载和推理器缓存降低重复开销
人脸增强	GFPGAN 会增加额外模型推理步骤	设计为可选开关，默认由用户按图片类型决定

从工程角度看，性能优化不应只追求单次最快速度，还应关注用户感知。健康检查让用户知道系统当前使用什么设备；模型列表让用户知道权重是否就绪；任务记录让用户保留不同模型的处理结果；滑块对比让用户判断更慢的模型是否带来足够收益。这些设计共同构成了面向用户的性能解释机制。

6.6 测试结论

综合功能测试、边界测试、数据集量化测试和性能测试可以看出，本系统已经形成本地图像超分应用的基本闭环。前端能够完成上传、参数配置、结果预览和任务记录；后端能够提供健康检查、模型列表和图像增强接口；推理核心能够复用多类模型并进行权重管理；启动脚本能够处理虚拟环境、端口和前端构建问题。这些结果说明系统达到毕业设计的主要目标。

需要说明的是，当前量化测试覆盖 Set5 和 Set14，能够反映系统在标准小规模数据集上的基本表现，但还不是完整的模型评测平台。原因在于本项目重点是系统设计与工程实现，而不是重新训练或全面比较超分模型。若后续继续完善，可以把当前测试过程封装为自动化评测脚本，扩展到 BSDS100、Urban100 和 Manga109，并增加 LPIPS、显存占用和批量报告导出等指标。

6.7 工程特点分析

本系统的主要工程特点有三点。第一，模型配置集中。所有模型信息集中在 `core.py`，前端通过 API 获取模型列表，降低了前后端配置不一致的风险。第二，推理接口统一。`Real-ESRGAN`、`SwinIR` 和 `GFPGAN` 的底层接口存在差异，但系统向前端暴露统一的增强接口，使页面交互稳定。第三，启动流程完整。脚本处理虚拟环境、端口、前端构建和浏览器打开，使系统更接近可交付应用。

同时，系统仍存在不足。首先，当前任务记录保存在浏览器本地，只适合单机使用；如果后续需要多用户或长期归档，应引入数据库或文件索引。其次，系统没有实现批量处理和任务队列，大图推理时前端只能等待单个请求完成。再次，模型权重自动下载依赖网络环境，若用户网络受限，需要手动放置权重文件。最后，当前量化测试规模仍有限，尚未覆盖 `LPIPS`、显存峰值和更多模型组合。

6.8 风险分析与改进方案

为了使系统评价更完整，本文进一步从工程风险角度总结当前实现的潜在问题和改进方向，如表 6.7 所示。

表 6.7 系统风险与改进方案

风险点	可能影响	改进方案
大图处理耗时长	用户等待时间增加，浏览器可能误以为无响应	引入异步任务队列和进度轮询，前端展示排队状态
本地任务记录有限	换浏览器或清理缓存后历史记录丢失	增加轻量文件索引或 <code>SQLite</code> 数据库保存任务元信息
权重下载依赖网络	演示现场可能因网络问题导致模型不可用	提供权重预检脚本和离线权重放置说明
评测规模有限	对不同图像类型和模型组合的覆盖仍不足	扩展批处理评测脚本，增加更多数据集和指标
单机服务边界固定	无法支持多人同时使用	后续若远程部署，应加入认证、队列、限流和文件隔离

这些风险并不影响当前系统作为毕业设计演示工具运行，但说明系统距离生产级图像增强平台仍有差距。把风险和改进方案写入论文，可以更清楚地体现毕业设计的边界意识，也能在答辩中说明哪些功能是本阶段完成的，哪些功能适合作为后续工作。

6.9 本章小结

本章从测试环境、功能测试、边界条件、数据集量化测试、性能测试和风险改进等方面验证了系统完成情况。测试分析表明，系统已经能够完成上传、推理、预览、下载、任务记录和环境检查等核心流程，并能对无效输入、权重缺失、端口占用和资源受限等情况给出处理思路。**Set5** 和 **Set14** 的测试结果进一步说明系统具备基本的定量评价能力，但批量评测平台、显存统计和任务队列仍适合作为后续改进方向。

7 总结与展望

7.1 工作总结

本文围绕“基于 CNN 的图像超分辨率系统设计与实现”完成了一个本地 Web 图像超分系统。系统以 Real-ESRGAN 为主要推理基础，扩展集成 SwinIR 与 GFPGAN，采用 Vue + Vite 构建前端交互界面，采用 FastAPI 封装后端推理服务，并通过统一启动脚本降低部署和使用复杂度。论文从研究背景、技术基础、需求分析、总体设计、系统实现和系统测试等方面对项目进行了完整说明。

从毕业设计完成度看，本项目不仅实现了模型推理调用，还围绕实际使用流程补齐了界面、参数、结果、历史、模型状态、异常处理和一键启动等工程能力。相比单纯运行推理脚本，该系统更便于演示、使用和维护，也更符合软件开发类毕业设计对系统性和完整性的要求。

7.2 后续展望

后续工作可以从三个方向展开。第一，扩展自动化评测模块。系统可基于 `test_images/` 中的标准数据集批量生成低分辨率输入，计算 PSNR、SSIM、LPIPS、推理延迟和显存占用，形成更完整的模型对比报告。第二，引入任务队列和批处理能力。对于多张图片或大尺寸图片，后端可以增加异步任务队列，前端显示排队状态和进度。第三，探索模型轻量化与部署优化。可以结合轻量化超分网络、ONNX、NCNN 或 Core ML 等推理后端，在保证质量的前提下降低普通设备上的推理成本。

总体而言，本系统已完成本地图像超分工具的核心闭环。随着模型评测、批量处理和部署优化能力的补充，系统可以进一步从毕业设计演示工具发展为更实用的个人图像增强应用。

参考文献

- [1] Jifei Yu, Xinbo Gao, Dacheng Tao, Xuelong Li, Kaibing Zhang. A Unified Learning Framework for Single Image Super-Resolution[J]. IEEE Transactions on Neural Networks and Learning Systems, 2014, 25(4): 780-792.
- [2] Chao Dong, Chen Change Loy, Kaiming He, Xiaoou Tang. Image Super-Resolution Using Deep Convolutional Networks[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2016, 38(2): 295-307.
- [3] Jiwon Kim, Jung Kwon Lee, Kyoung Mu Lee. Accurate Image Super-Resolution Using Very Deep Convolutional Networks[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 1646-1654.
- [4] Yulun Zhang, Kunpeng Li, Kai Li, Lichen Wang, Bineng Zhong, Yun Fu. Image Super-Resolution Using Very Deep Residual Channel Attention Networks[C]//Proceedings of the European Conference on Computer Vision. 2018: 286-301.
- [5] Christian Ledig, Lucas Theis, Ferenc Huszar, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, Wenzhe Shi. Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2017: 4681-4690.
- [6] Xintao Wang, Ke Yu, Shixiang Wu, Jinjin Gu, Yihao Liu, Chao Dong, Yu Qiao, Chen Change Loy. ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks[C]//Proceedings of the European Conference on Computer Vision Workshops. 2018.
- [7] Xintao Wang, Liangbin Xie, Chao Dong, Ying Shan. Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops. 2021: 1905-1914.
- [8] Jingyun Liang, Jiezhang Cao, Guolei Sun, Kai Zhang, Luc Van Gool, Radu Timofte. SwinIR: Image Restoration Using Swin Transformer[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops. 2021: 1833-1844.
- [9] Chao Dong, Chen Change Loy, Xiaoou Tang. Accelerating the Super-Resolution Convolutional Neural Network[C]//Computer Vision – ECCV 2016. 2016: 391-407.
- [10] Wenzhe Shi, Jose Caballero, Ferenc Huszar, Johannes Totz, Andrew P. Aitken, Rob Bishop, Daniel Rueckert, Zehan Wang. Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 1874-1883.
- [11] Bee Lim, Sanghyun Son, Heewon Kim, Seungjun Nah, Kyoung Mu Lee. Enhanced Deep Residual Networks for Single Image Super-Resolution[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops. 2017: 136-144.
- [12] Muhammad Haris, Greg Shakhnarovich, Norimichi Ukita. Deep Back-Projection Networks for Super-Resolution[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 1664-1673.
- [13] Justin Johnson, Alexandre Alahi, Li Fei-Fei. Perceptual Losses for Real-Time Style Transfer and Super-Resolution[C]//Computer Vision – ECCV 2016. 2016: 694-711.

- [14] Pengxu Wei, Ziwei Xie, Guanbin Li, Liang Lin. Taylor Neural Network for Real-World Image Super-Resolution[J]. IEEE Transactions on Image Processing, 2023, 32: 1942-1951.
- [15] Yao Fan, Min Jiang, Zhaoyang Liu, Zhiwei Zhao, Lizhi Zheng. Lightweight Image Super-Resolution Reconstruction Guided by Gated Feedback and LatticeFormer[J]. IEEE Access, 2025, 13: 53214-53226.
- [16] Axi Niu, Kang Zhang, Trung X. Pham, Jinqiu Sun, Yu Zhu, In So Kweon, Yanning Zhang. CDPMSR: Conditional Diffusion Probabilistic Models for Single Image Super-Resolution[C]//Proceedings of the IEEE International Conference on Image Processing. 2023: 615-619.
- [17] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, Eero P. Simoncelli. Image Quality Assessment: From Error Visibility to Structural Similarity[J]. IEEE Transactions on Image Processing, 2004, 13(4): 600-612.
- [18] Kai Zhang, Jingyun Liang, Luc Van Gool, Radu Timofte. Designing a Practical Degradation Model for Deep Blind Image Super-Resolution[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2021: 4791-4800.
- [19] Xintao Wang, Yu Li, Honglun Zhang, Ying Shan. Towards Real-World Blind Face Restoration with Generative Facial Prior[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2021: 9168-9178.
- [20] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library[C]//Advances in Neural Information Processing Systems: vol. 32. 2019.
- [21] Pablo Arbelaez, Michael Maire, Charless Fowlkes, Jitendra Malik. Contour Detection and Hierarchical Image Segmentation[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2011, 33(5): 898-916.

致谢

本论文和毕业设计的完成，离不开老师、同学和家人的帮助。

首先感谢李建华老师在课题方向、系统设计和论文写作过程中给予的指导。从最初确定图像超分辨率系统的实现边界，到后续梳理模型推理、前后端交互和工程测试的关系，老师的建议帮助我避免了只停留在模型调用层面的实现方式，使本课题能够形成较完整的软件系统。

感谢计算机与信息学院在本科阶段提供的课程训练和实验环境。数据结构、计算机网络、软件工程、人工智能和 Web 开发等课程，为本项目中的接口设计、模块拆分、异常处理和系统测试奠定了基础。毕业设计也让我更直接地体会到，深度学习应用并不只依赖模型本身，工程封装、用户交互和可维护性同样重要。

感谢同学们在项目调试和界面体验方面提出的意见。图像上传、模型选择、结果对比、任务记录和一键启动等功能，都是在多次试用和修改后逐步完善的。感谢开源社区提供的 Real-ESRGAN、SwinIR、GFPGAN、PyTorch、FastAPI 和 Vue 等工具与资料，这些成果为本系统的实现提供了重要基础。

最后感谢家人在学习和生活上的支持。毕业设计是本科阶段一次较完整的综合训练，也暴露出我在模型评测、性能优化和论文表达方面仍有不足。今后我会继续保持严谨、务实的学习态度，在工程实践中不断提升解决问题的能力。

作者：张磊

2026 年 5 月 18 日

附录

附录 A 系统启动命令

项目提供 Python 和 Windows 批处理两类启动入口。常用命令如下：

Listing 1 系统启动与构建命令

```
python 启动超分界面.py
python 启动超分界面.py --port 7860
python 启动超分界面.py --strict-port
python 启动超分界面.py --rebuild-frontend
python 启动超分界面.py --skip-frontend-build
python 启动超分界面.py --gradio
```

默认情况下，系统会优先启动 Vue + FastAPI 新界面；当需要快速验证推理核心或前端构建环境异常时，可以通过 `--gradio` 参数启动旧版 Gradio 界面。若端口被占用且未指定 `--strict-port`，启动脚本会自动尝试后续端口。

附录 B 图像增强接口参数

表 7.1 图像增强接口主要参数

参数名	类型	说明
file	文件	用户上传的输入图片，支持常见 PNG、JPG、JPEG 和 WebP 格式
model	字符串	模型键，对应 MODEL_CONFIGS 中的模型配置
outscale	浮点数	输出倍率，默认使用模型配置中的倍率，也可由用户调整
denoise_strength	浮点数	降噪强度，主要用于 realesr-general-x4v3
tile	整数	分块大小，数值越小显存压力越低，但推理时间可能增加
tile_pad	整数	分块边缘填充，用于缓解块边界伪影
face_enhance	布尔值	是否启用 GFPGAN 人脸增强
fp32	布尔值	是否强制使用 FP32 精度，适合半精度不稳定的设备

附录 C 项目目录说明

表 7.2 项目主要目录和文件

路径	说明
启动超分界面.py	本地一键启动脚本，负责环境切换、端口选择、前端构建和服务启动
Real-ESRGAN/ core.py	共享推理核心，集中管理模型配置、权重下载、设备选择和结果保存
Real-ESRGAN/ vue_server.py	FastAPI 后端入口，提供健康检查、模型列表和图像增强接口
Real-ESRGAN/ app.py	旧版 Gradio 入口，复用 core.py 推理逻辑
frontend/src/ App.vue	Vue 前端主组件，包含工作台、模型管理和任务记录页面
frontend/dist/ test_images/	前端生产构建产物，由后端作为静态资源提供测试图片集合，用于功能验证和指标评测